


SEMESTER END EXAMINATIONS - JULY / AUGUST 2025

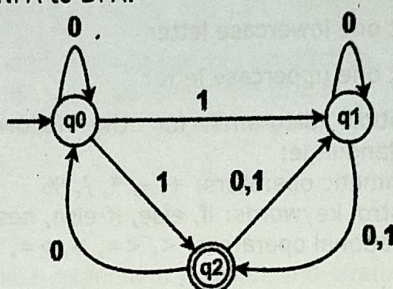
Program	: B.E. - CSE (Cyber Security) / CSE (Artificial Intelligence and Machine Learning)	Semester	: IV
Course Name	: Automata Theory and Compiler Design	Max. Marks	: 100
Course Code	: 23CY44 / 23CI44	Duration	: 3 Hrs.

Instructions to the Candidates:

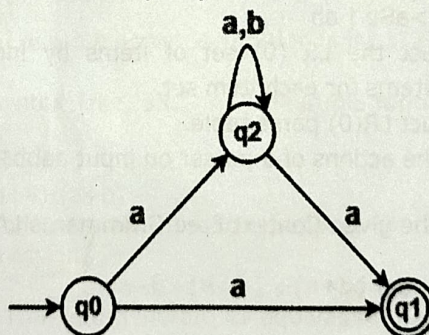
- Answer one full question from each unit.

UNIT - I

- Design DFAs to accept the following languages.
 - String beginning with 'bba' over $\Sigma = \{a, b\}$. CO4 (09)
 - String not ending with 01 over $\Sigma = \{0, 1\}$.
 - String starting with 'aa' and ending with 'bb'.
 - Convert the given NFA to DFA. CO4 (07)



- Describe the components for constructing a NFA. Construct an NFA with $\Sigma = \{0, 1\}$ that accepts all strings that begin with a and end with ab. CO4 (04)
- Explain the analysis and synthesis phase of the compiler by translating the assignment statement. Assume all variables are integers. CO1 (08)
 $Res = a * a + 2 * a * b + b * b;$
 - Design an NFA with $\Sigma = \{0, 1\}$ which accepts the language consists of all the strings containing substring 1010. CO4 (05)
 - Convert the following Non-Deterministic Finite Automata (NFA) to Deterministic Finite Automata (DFA). Construct the Transition Table. CO4 (07)


UNIT - II

- How does a lexical analyzer employing a two-buffer scheme with sentinel characters process the lexeme $a \leq b$ in a source line, and what benefits does this method provide in efficiently handling look-ahead scenarios? CO2 (06)

- b) Describe the language accepted by the regular expression: CO2 (06)
- $(a|b)aab^*$
 - $(ab|ba)^*$
 - $(a|b)^*b(a|b)$
- c) Construct the pushdown automata for accepting following languages: CO2 (08)
- $\{a^n b^n \mid n > 0\}$
 - $\{a^n b^{2n} \mid n > 0\}$
4. a) How does the separation of lexical analysis from parsing influence the clarity, modularity, and maintainability of a compiler, and how do the roles of tokens, patterns, and lexemes contribute to this design choice? CO2 (06)
- b)
 - Write a regular expression to validate a simple email address format. CO2 (05)
 - Construct a regular expression that enforces the following password rules:
 - At least one digit
 - At least one lowercase letter
 - At least one uppercase letter
- c) Design transition diagrams for the following components of a programming language: CO2 (09)
- Arithmetic operators: $+$, $-$, $*$, $/$, $\%$
 - Control keywords: if, else, if-else, nested if, and nested for
 - Relational operators: $<$, $<=$, $>$, $>=$, $==$, $!=$
- UNIT - III**
5. a) Consider the following Context Free Grammar: CO3 (10)
- $S \rightarrow SA \mid 0 \mid \epsilon$
 $A \rightarrow aS1 \mid a$
- Compute the FIRST and FOLLOW sets for each non-terminal symbol.
 - Construct the parsing table for a non-recursive predictive parser for the above grammar.
 - Is the grammar LL (1)? Justify.
- b) Consider the grammar CO3 (10)
- G:- $S \rightarrow aSb \mid ab$
- Construct the LR (0) set of items by indicating the Kernel and Non-kernel items for each item set.
 - Construct LR(0) parse table.
 - Show the actions of a parser on input aabb\$
6. a) Prove that the given Context Free Grammar is LALR(1) by generating the parse table. CO3 (10)
- $S \rightarrow Aa \mid bAc \mid dc \mid bda$
 $A \rightarrow d$
- b) Construct LR(0) parse table for the Grammar Given CO3 (10)
- G: $S \rightarrow L$
- $L \rightarrow (L)L \mid a \mid \epsilon$
- Do parsing for the input: (a)\$

23CY44/23CI44

UNIT- IV

7. a) Design a syntax-directed definition (SDD) using only synthesized attributes to evaluate arithmetic expressions involving digits, the operators +, *, and parentheses for the following grammar. CO4 (10)
- $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{digit}$
- i) Include semantic rules to compute the val attribute for each nonterminal.
ii) Use digit.lexval to refer to the integer value of a digit, as returned by the lexical analyzer.
- Then, using your SDD, construct the annotated parse tree for the input expression: $3 * 5 + 4$.
- b) Obtain a Turing machine to accept the language. CO4 (10)
- $L = \{0^n 1^n \mid n \geq 1\}$
8. a) Given the production and semantic rule: CO4 (10)
- $E \rightarrow E_1 + T$ with $E.val = E_1.val + T.val$ and the following values from a parse tree:
 $E_1.val = 10$
 $T.val = 5$
- (i) Draw the dependency graph for this production.
(ii) Compute the value of $E.val$ using the graph.
(iii) Extend the production to a larger expression:
 $E \rightarrow T$
 $T \rightarrow T_1 + F$
 $T \rightarrow F$
 $F \rightarrow \text{digit}$
- Construct a complete dependency graph and evaluate the final value of $E.val$.
- b) Describe the language of a Turing Machine. Specify the Turing Machine and halting problem. CO4 (10)

UNIT - V

9. a) Translate the following control flow statement with the help of semantic actions of one pass code generation. CO5 (10)
- if($a < b$) $a = 1$; else $a = 0$;
- b) Translate the following array reference assignment statements with the help of SDT. CO5 (10)
- i) $x = a[i] + b[j]$
ii) $c = a + b[i][j]$ assume array declaration is $\text{int } b[2][2]$
10. a) Construct DAG, Syntax Tree, 3AC and Quadruple representation for CO5 (10)
- i. $x = a + b * -c + b * -c$
ii. $x = a + b + b + a + a * b$
iii. $x = (a + b) * c + (d + a) - (a + b)$
- b) Construct DAG and generate its corresponding three address code for the following expression. CO5 (10)
- $a = a + b + (a + b) + (a + b)$
- Draw the Syntax Tree and generate its equivalent three address code for the same expression. Does both generate the same three address code sequence? If not, state the reason.
